Lab 8: Stream Processing (Part 2)

Our goal is to learn the general idea behind building a streaming application with Spark+Kafka and give a look at its main concepts using real data.

Kafka and Spark in a Nutshell

The idea is simple – Apache Kafka is a message streaming tool, where producers write messages on one end of a queue (called a topic) to be read by consumers on the other.

But it's a very complex tool, built to be a resilient distributed messaging service, with all sorts of delivery guarantees, message storage, and message replication, while also allowing flexibility, scalability, and high throughput.

Apache Spark is a distributed memory-based data transformation engine.

It's also a very complex tool, able to connect with all sorts of databases, file systems, and cloud infrastructure. It is geared to operate in distributed environments to parallelize processing between machines, achieving high-performance transformations by using its lazy evaluation philosophy and query optimizations.

Join these two technologies and we have a perfect match to build a streaming ETL pipeline.

The application

We'll be using the data from traffic sensors in the city of Belo Horizonte (BH), the capital of Minas Gerais (Brazil). It's a huge dataset containing measurements of traffic flow in several places in the city. Each sensor periodically detects the type of vehicle driving at that location (car, motorcycle, bus/truck), its speed and length, and other information.

This dataset represents precisely one of the classical applications for streaming systems – a group of sensors sending their readings continuously from the field.

In this scenario, Apache Kafka can be used as an abstraction layer between the sensors and the applications that consume their data.

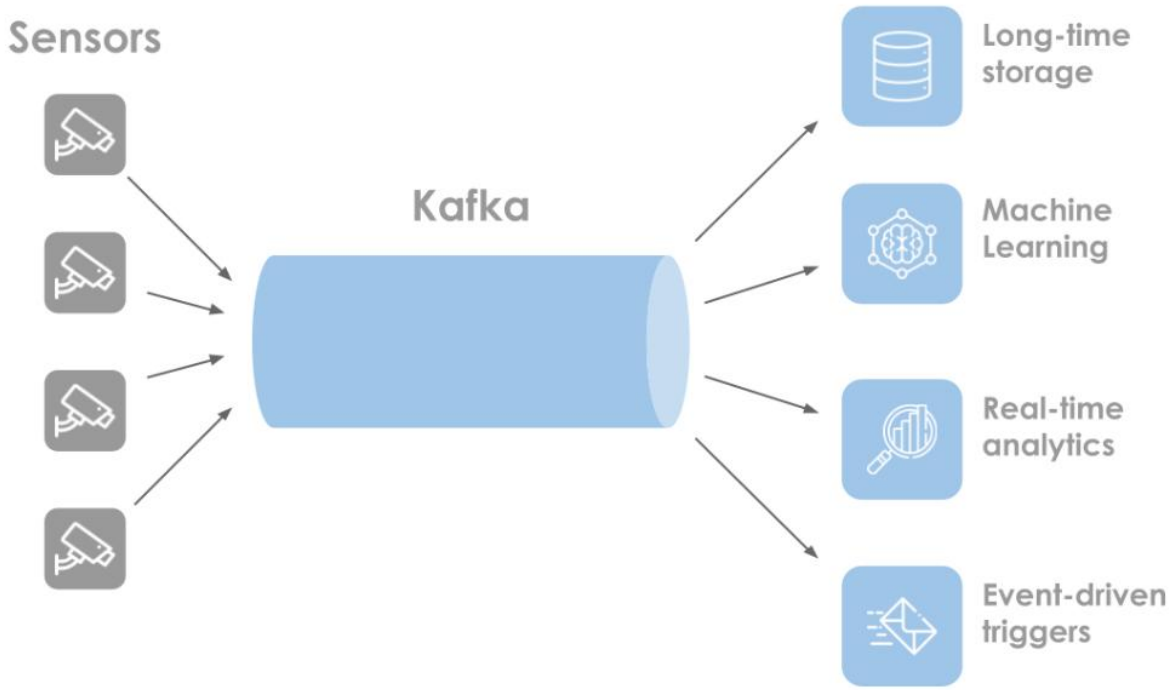

Figure: Kafka used as an abstraction layer between sources and services.

With this kind of infrastructure, it's possible to build all sorts of (the so-called) real-time event-driven systems, like a program to detect and alert for traffic jams when the number of vehicles suddenly increases with a drop in average speed.

And that's where Apache Spark comes into play.

It has a native module for stream processing called *Spark Structured Streaming*, that can connect to Kafka and process its messages.

Setting up the environment

We'll use a docker-compose file.

The **./src** volume is where we going to put our scripts.

To start the environment, just run:

```
docker-compose up
```

The implementation

Spark's streaming module is very similar to the usual spark code, making it easy to migrate from the batch applications to the stream ones.

Our first job

The first thing to do is create a Kafka topic from where our spark job will consume the messages.

Question 1:

Create a Kafka topic called test_topic.

Produce messages to the test_topic, where the key and value are separated by a colon (:).

From now, every line typed in the terminal will be sent as a message to the test_topic. The character ":" is used to separate the message's key and value (key:value).

Let's create a Spark job to consume this topic.

Question 2: Create a PySpark Structured Streaming application that reads data from the topic **test_topic**. The application should consume messages starting from the earliest available offset and output the results to the console.

The code needs to be put inside the **/src/streaming** folder.

The key thing to note is that we're using the attributes **readStream** and **writeStream**, instead of normal read and write. This is the main aspect that makes Spark treats our job as a streaming application.

To connect to Kafka, it is necessary to specify the server and topic. The option **startingOffsets="earliest"** tells Spark to read the topic from the beginning. Also, because Kafka stores its messages in binary form, they need to be decoded to string.

Now, let's access the Spark container and run the job.

```
spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.3.0
/src/streaming/read_test_stream.py
```

After a few seconds, it will start consuming the topic.

Spark Streaming works in micro-batching mode, and that's why we see the "batch" information when it consumes the messages.

In the context of **Spark Streaming**, **micro-batching** is an approach used to handle **streaming data** in small, manageable chunks (or batches), rather than processing the data in a continuous, real-time manner.

Writing data to a Kafka stream

Now it's time to start playing with the sensor data.

Extract the **AGOSTO_2022.zip** file into the **/data** volume. The data is originally in JSON. The first thing to do is convert it to **parquet** to optimize disk space and reading time.

All you need to do is to execute the following spark jobs:

```
spark-submit /src/transform_json_to_parquet.py
```

```
spark-submit /src/join_parquet_files.py
```

Depending on your machine, the execution may take some time.

We also need to create the Kafka topic to receive our messages:

Question 3: Create a Kafka topic named traffic_sensor with a replication factor of 1 to receive messages.

To display the arriving messages, we use a console consumer.

Question 4: Run a console consumer to display the arriving messages from the **traffic_sensor** topic using the appropriate Kafka command.

There are some details to consider when writing data to a Kafka topic.

In structured streaming, the default behavior is to not try to infer the data schema (columns and their types), so we need to pass one.

Kafka messages are just key-value binary string pairs, so we need to represent our data in this format. This can be easily achieved by converting all rows to JSON strings, encoding them in binary, and storing the result in the "value" column.

Name	Age	Country
João	21	Brazil
Paul	22	USA

→

key	value
key	{name:João, age:21, country:Brazil}
key	{name:Paul, age:22, country:USA}

Figure: Transforming columns to JSON strings.

Message keys are very important in Kafka, but they will not be useful in our tests, so all messages will have the same.

Question 5:

Write a PySpark application that reads data from a Parquet file, processes it, and streams it to a Kafka topic. The following requirements must be met:

1. **Input Data**:
    - Read data from a Parquet file located at /data/AGOSTO_2022_PARQUET_FINAL/.
    - The schema of the data includes the following fields:
        - ID EQP (Equip. ID): LongType

- DATA HORA (Date and hour): TimestampType
- MILESEGUNDO (Millissecond): LongType
- CLASSIFICAÇÃO (Classification: Motocycle, Car, bus/truck, etc.): StringType
- FAIXA (Road lane): LongType
- ID DE ENDEREÇO (Address ID): LongType
- VELOCIDADE DA VIA (Road speed): StringType
- VELOCIDADE AFERIDA (Measured vehicle speed): StringType
- TAMANHO (Vehicle length): StringType
- NUMERO DE SÉRIE (Serial number): LongType
- LATITUDE: StringType
- LONGITUDE: StringType
- ENDEREÇO (Address): StringType
- SENTIDO (Direction): StringType

2. **Kafka**:
   - Write the processed data to the Kafka topic **traffic_sensor**.
3. **Stream Processing**:
   - Convert each record into a JSON string and assign a static (fixed) key for each message.
   - Ensure that the message key and value are encoded in iso-8859-1 and cast to binary format.
4. **Checkpointing**:
   - Use /tmp/checkpoint as the checkpoint location for your stream.
5. **Limit**:
   - Limit the number of records to be processed to 500,000.

As mentioned before, this dataset is HUGE, so we limited the number of messages inserted to 500,000.

Finally, we pass the Kafka server and topic and a "checkpointLocation" where the spark will store the execution progress, useful to recover from errors.

Question 6: Execute this job.

Our traffic topic is populated and almost ready to be processed.

It's important to remember that we used a spark job to populate our topic just for learning purposes. In a real scenario, the sensors themselves will send readings directly to Kafka.

To simulate this dynamic behavior, we will create a script that writes one row to the topic every 2.5 seconds.

Question 7:

Modify the PySpark application to read data from a Parquet file and write it to a Kafka topic using **batch processing** instead of stream processing. The requirements are as follows:

1. **Batch Processing:**
    - Instead of reading and processing data in real-time (streaming), read the data in batch mode.
    - Process one record at a time and write it to the Kafka topic traffic_sensor.
2. **Data Conversion:**
    - Convert each record to JSON format.
    - Assign a fixed (static) key for each message.
3. **Write Records:**
    - Write each processed record to the Kafka topic **one by one**.
4. **Batch Logic:**
    - Introduce a delay of **2.5 seconds** between each record write operation.
5. **Kafka Integration:**
    - Ensure that the processed data is written to the Kafka topic traffic_sensor.

Notes:

- Instead of using the **writeStream** method, use the **write** method to output the data in batches.
- Limit the number of records processed to **50,000**.

Output modes – Counting the number of vehicles by type

Moving on, let's create a job to count the number of vehicles by type.

The column "Classificação" (Classification) contains the vehicle type detected.

Question 8: Create a PySpark Structured Streaming application that count the number of vehicles by type. The output written to the console must be updated continuously as new data arrives.

The output mode of a stream application specifies how we want to (re)compute and write the results as new data arrives.

It can assume three different values:

- **Append:** Only add new records to the output.
- **Complete:** Recompute the full result for each new record.
- **Update:** Update changed records.

The "append" mode is not available to queries with grouping.

Tumbling time window – Aggregating using time intervals

Tumbling windows are non-overlapping fixed time intervals used to make aggregations. To put it more simply, they slice the timeline into equally sized slices so each event belongs to a single interval.

For example, count, every 5 minutes, how many vehicles were detected.

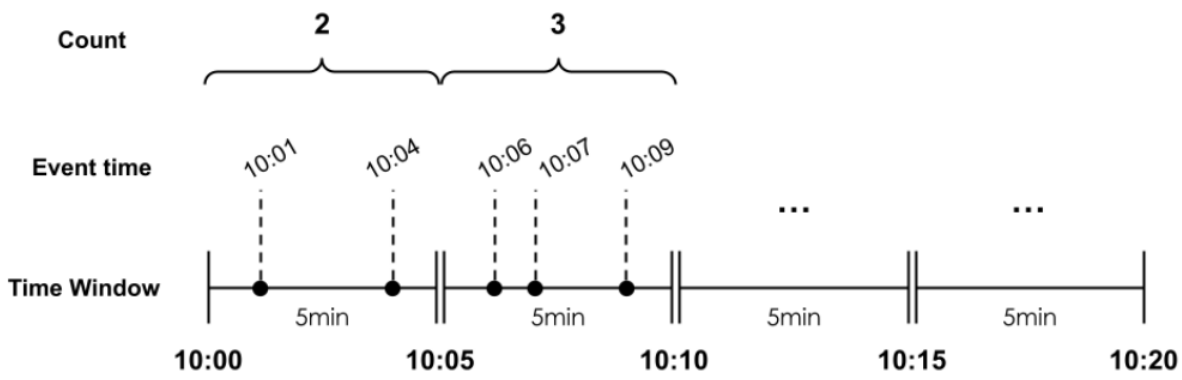

Figure: 5 min tumbling window.

Question 9: Create a PySpark Structured Streaming application that counts the total number of records in a 5-minute tumbling window based on the **DATA HORA** timestamp.

Time windows can also be used in conjunction with other columns in the groupBy().

Question 10: Create a PySpark Structured Streaming application that counts the number of vehicles by type in a 5-minute tumbling window based on the **DATA HORA** timestamp.

Sliding time window – Flexibilization on the time intervals

Sliding time windows are a variation of tumbling windows that create overlapping intervals.

For example, every 5 minutes, count how many vehicles were detected in the last 30 minutes.

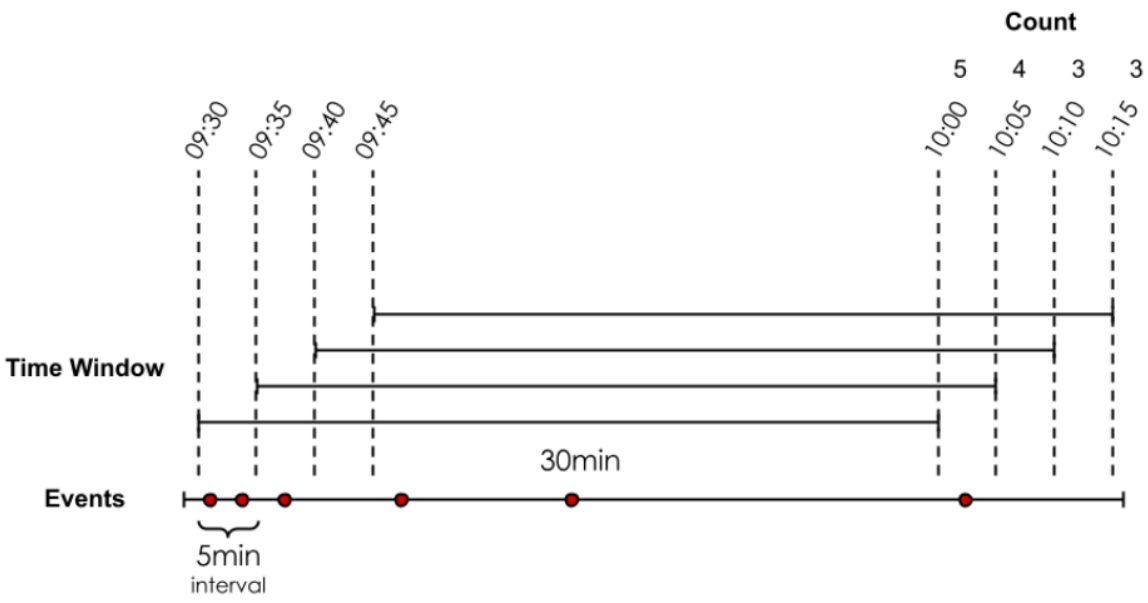

Figure: sliding window.

Because of that, events can belong to many intervals.

Question 11: Create a PySpark Structured Streaming application that counts the total number of records in a 30-minute sliding window, with a new window created every 5 minutes, based on the **DATA HORA** timestamp. The output should be displayed in the console without truncating the results, and it should update continuously as new data arrives.

Conclusion

This was an overview at the main concepts of Spark Structured Streaming and how they can be applied with Kafka.

Apache Kafka and Apache Spark are both reliable and robust tools used by many companies to daily process incredible amounts of data, making them one of the strongest pairs in the stream processing task.